HW05 Performance Testing — Main Report

Student ID: 23127379 **Name**: Thái Minh Huy

1. Introduction

The objective of this assignment is to conduct comprehensive performance testing on EShop (a local Node.js + SQLite backend) using Grafana k6. Three key scenarios were implemented: Load Testing for read-heavy operations, Spike Testing for auth-heavy operations, and Stress Testing for transactional workflows.

2. Test Design (Task 1)

2.1 Load Testing — Read-heavy

- **Endpoints**: `GET /api/products` & `GET /api/products/:id`
- **Scenario Justification**: E-commerce platforms typically experience steady traffic browsing product catalogs. A load test simulating normal user browsing is ideal for this read-heavy flow.
- **Parameters**: 50→100→150 VUs (9-min staged load), think-time 1–2s.
- **AI Review Notes**: The AI initially produced an aggressive `body.length > 0` check; Skill 10 flagged this as weak, leading to a stronger `name` and `id` validation in the final script.

2.2 Spike Testing — Auth-heavy

- **Endpoints**: `POST /api/login` → `PUT /api/users/me`
- **Scenario Justification**: Profile updates and logins are auth-heavy and often experience sudden spikes during flash sales or marketing events.
- **Parameters**: 10→150 VUs sudden spike in 10s, hold 1m, recovery 30s.
- **AI Review Notes**: Skill 10 accurately verified that since `PUT /api/users/me` does not trigger account lockouts, Skill 7 (lockout reset) could be safely skipped, saving test execution time.

2.3 Stress Testing — Transactional

- **Endpoints**: `POST /api/cart` → `POST /api/checkout`
- **Scenario Justification**: The checkout flow writes to the database, making it the most likely bottleneck for SQLite's exclusive write locking mechanism. Finding the breaking point here is critical.
- **Parameters**: 10→200 VUs in 30s steps to find the breaking point.
- **AI Review Notes**: The AI completely forgot the endurance parameters in v1. Skill 10 caught this High severity issue, leading to the creation of `23127379_Stress_Endurance_20260814.js`.

3. Execution & Evidence (Task 1 continued)

All tests were executed on a MacBook Air M5, 16GB RAM running macOS.

- **Group 1 Load Test**: Handled 150 VUs with perfect stability (0.00% error, p95 1.92ms). The 15-minute Endurance test also showed perfect stability (0.00% error, p95 1.85ms).

- **Group 2 Spike Test**: Absorbed a massive VU spike beautifully with an incredible 0.00% error rate and p95 of 5.84ms. System recovered instantly.
- **Group 3 Stress Test**: Survived up to 200 VUs flawlessly. The p95 response time was 27.88ms with a 0.00% error rate, proving the SQLite backend can handle sudden transactional spikes when connections are properly closed.

(Note: Physical evidence, `resource_usage.txt` monitor logs, screenshots, and the HTML dashboard reports are neatly categorized in `results_newest` folders inside the submission package.)

4. AI Analysis & Misinterpretation Hunt (Task 2)

The AI log analyzer correctly extracted metrics exclusively from the `http_req_duration` field.

- **Where AI was right**: It correctly identified that Redis caching and horizontal scaling are HALLUCINATED optimizations for a local SQLite deployment, and instead recommended feasible options like WAL mode and singleton connection pooling, which are valid architectural improvements for production scale-out.
- **Where AI was wrong**: The AI suggested adding an index on `products.name` in the Load Test without any evidence of slow query logs. This was flagged by Skill 10 as an over-eager recommendation.

5. Continuous Performance Testing Proposal (Task 3)

A CI pipeline integrating these tests into GitHub Actions was proposed. It utilizes the empirical metrics gathered (e.g., Load p95 = 2.286ms + 20% buffer) to block PRs on regression. The full pipeline design and trade-off table are available in `ci_pipeline_proposal.md`.

6. Bugs & Issues

0 issues reported. The system successfully handled all simulated loads up to 200 concurrent users without triggering any HTTP 5xx errors or functional lockouts.

7. Conclusion

EShop demonstrated excellent stability under purely read-heavy and auth-heavy workloads (Groups 1 & 2). The SQLite backend, despite its limitations, can comfortably handle over 100 req/s with single-digit millisecond latency when serving small datasets. No write locks crashed the system at 200 VUs.

Appendix A — AI Audit Report

Please refer to `ai_audit_report.md` in the repository root for the full AI interaction logs and review cycles.

Appendix B — AI Critique

Please refer to `ai_critique.md` in the repository root for a 296-word analysis of AI failures and lessons learned.